MEDAC
Instituto Oficial de Formación Profesional

PROGRAMACIÓN DE INTELIGENCIA ARTIFICIAL

Tipos de lenguajes de programación

2

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Lenguajes de bajo nivel 4

/ 3. Lenguajes de alto nivel 4

/ 4. Caso práctico 1: “Un intérprete para problemas de espacio” 5

/ 5. Lenguajes por paradigma de programación 6

/ 6. Lenguajes de marcado 7

/ 7. Caso práctico 2: “La base del *web scraping*” 8

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 9

© MEDAC
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer la diferencia entre lenguajes de alto nivel y bajo nivel.

Descubrir los diferentes paradigmas de programación y los tipos de programación asociados a ellos.

Introducir los lenguajes de marcado.

/ 1. Introducción y contextualización práctica

En la anterior unidad, hicimos una primera introducción a los lenguajes de programación, analizando su definición y repasando la historia a través de algunos de los lenguajes más famosos.

En esta unidad, ahondaremos un poco más, analizando los diferentes tipos de lenguajes de programación y sus respectivas características, ventajas e inconvenientes.

Además, estudiaremos los tipos de lenguajes de programación según sus fines. En particular, nos centraremos en los lenguajes de marcado y los lenguajes multiplataforma.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema, encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Fig.1. ¿Qué tipos de lenguajes de programación existen?

Audio intro. "La elección del lenguaje"
https://bit.ly/3mkKvne

/ 2. Lenguajes de bajo nivel

Tradicionalmente, los lenguajes de programación se han dividido en dos grandes tipos, dependiendo de su nivel de abstracción respecto a las características físicas de las máquinas (hardware): **lenguajes de bajo nivel y alto nivel**.

Los lenguajes de bajo nivel están orientados y condicionados por el hardware. Tratan de aprovechar todas las características de este, creando así un vínculo íntimo entre el lenguaje (software) y la máquina (hardware). Por esta razón, a diferencia de los lenguajes de alto nivel, los lenguajes de bajo nivel con frecuencia son crípticos o no legibles por humanos. Además, dado que estos lenguajes están vinculados a una máquina determinada, normalmente no son transportables de una máquina a otra.

Dentro de esta categoría, podemos destacar una segunda subcategoría: **lenguaje de máquina y lenguaje ensamblador**.

El **lenguaje de máquina** es el más primitivo de los lenguajes y es directamente entendible por la máquina, constituyendo una colección de dígitos binarios o bits en forma de 0 y 1 (por ejemplo: 101100010001010). El lenguaje de máquina es muy difícil de comprender por el humano, además de ser muy propenso a errores y difícil de mantener.

El **lenguaje ensamblador** es el primer intento de remediar los problemas de desarrollo, mantenimiento e interpretabilidad del lenguaje máquina. Los programas escritos en lenguaje ensamblador están almacenados como texto, representando un flujo de instrucciones ejecutables por un microprocesador. El lenguaje ensamblador cuenta con comandos básicos en inglés (*mov, add, sub*...), lo que facilita la comprensión de las instrucciones.

Para que estas instrucciones sean entendidas por la máquina, deben ser convertidas a lenguaje máquina mediante un programa llamado **ensamblador**.

Investigamos...

A continuación, te recomendamos buscar en Internet más información sobre el tema de lenguaje de máquina, la base para el funcionamiento de los ordenadores modernos.

/ 3. Lenguajes de alto nivel

El objetivo de los **lenguajes de alto nivel** es facilitar el trabajo de los programadores, empleando instrucciones fáciles de entender.

Los lenguajes de programación de alto nivel presentan un nivel de abstracción respecto al hardware mucho mayor, permitiendo escribir código a través de un conjunto de palabras o instrucciones provenientes de idiomas humanos (normalmente en inglés).

Además, estos idiomas presentan una serie de operadores lógicos, funciones y estructuras de control que permiten la creación de programas fácilmente entendibles por los programadores.

Los lenguajes de alto nivel son, por tanto, mucho más fáciles de leer, escribir y mantener que los de bajo nivel. Además, debido a su alto nivel de abstracción, los programas escritos en estos lenguajes son portables y compartibles.

Al igual que el lenguaje ensamblador, los lenguajes de alto nivel tienen que ser posteriormente traducidos al lenguaje de máquina. Este proceso se realiza a través de **los traductores (o intérpretes) y compiladores**:

- **Traductor o intérprete**: Traducen los programas a lenguaje de máquina a medida que se van ejecutando. La traducción se realiza cada vez que se ejecuta el programa. Algunos ejemplos de lenguajes que utilizan traductores son **Java**, **JavaScript** o **Python**.

- **Compilador**: Los compiladores traducen los programas a lenguaje de máquina solo una vez, lo cual permite una ejecución más rápida. A diferencia de los traductores, los compiladores crean un programa equivalente en lenguaje de máquina que, una vez compilado, se puede ejecutar tantas veces como se quiera, en diferentes máquinas. Algunos ejemplos son **C** y **Go**.

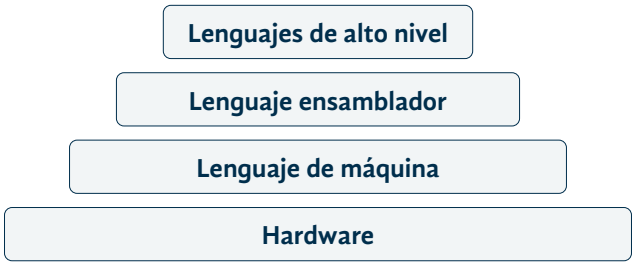

Fig.2. Nivel de abstracción de los lenguajes de programación.

Vídeo 1. "Sobre intérpretes y compiladores"

https://bit.ly/3sT4nC7

/ 4. Caso práctico 1: "Un intérprete para problemas de espacio"

Planteamiento: Hoy, 18 de junio de 1968, es el dieciocho cumpleaños de Sharon. Recientemente Sharon ha recibido la noticia de que ha sido admitida para estudiar Matemáticas en una prestigiosa escuela de Nueva York. Sus padres, anticipando la carrera brillante que le espera a su hija, deciden regalarle, además del último vinilo de los Beatles, un ordenador.

Nudo: Sharon es una apasionada de la informática. Ha dedicado muchos fines de semana a aprender FORTRAN por su cuenta. Con su nuevo ordenador, es el momento de poner en práctica lo aprendido. Sin embargo, su nuevo ordenador, a pesar de ser lo más nuevo del mercado, tiene muy poca memoria, lo que supone grandes problemas para compilar sus programas escritos en FORTRAN.

Desenlace: Harta de las limitaciones, Sharon decide prescindir de FORTRAN y pasarse a otro lenguaje de programación que venía preinstalado en su ordenador: Basic.

A diferencia de FORTRAN, Basic funciona con un intérprete, de tal forma que solo tiene que traducir sus programas a lenguaje de máquina a medida que los va ejecutando. De esta forma, aunque la ejecución es más lenta que si tuviera el programa compilado y debe ejecutarlo cada vez que enciende el ordenador, los problemas de memoria han desaparecido. Además, Sharon descubre que, utilizando un intérprete, puede interactuar con su código de forma más flexible, lo que le permite hacer modificaciones o mejoras de su código sobre la marcha.

La experiencia de Sharon fue vivida por muchos desarrolladores en los inicios de la informática. La limitada memoria de los primeros ordenadores personales hizo que se pusieran de moda los lenguajes de programación basados en traductores. Años más tarde, con el progreso de la tecnología y la mejora de los ordenadores, los lenguajes de alto nivel basados en compiladores ganaron peso, si bien muchos de los lenguajes de programación más populares en la actualidad se basan en intérpretes.

/ 5. Lenguajes por paradigma de programación

Los lenguajes también se pueden clasificar en función del paradigma de programación que emplean. Se entiende por **paradigma de programación** un marco conceptual o conjunto de principios que guían la construcción de un programa o software. Existen dos grandes paradigmas: **programación imperativa y declarativa**.

- **Programación imperativa**: Es el paradigma de programación clásico. Se centra en el 'cómo'. De acuerdo con este paradigma, un programa es una secuencia de instrucciones claramente definidas. Los lenguajes basados en este paradigma son muy precisos y fácilmente comprensibles, pero requieren muchas líneas de código para llegar a la solución de problemas. Algunos de los lenguajes basados en programación imperativa son FORTRAN, C, C++, Java o Python. Dentro de esta programación existen tres subtipos: **programación estructurada, procedimental y modular**.
  - » **Programación estructurada**: Contiene estructuras de control como secuencias, condicionales y bucles que permiten reducir la complejidad del código.
  - » **Programación procedimental**: Se basa en dividir las tareas en procedimientos parciales o funciones que se llaman cada vez que tengan que ejecutarse.
  - » **Programación modular**: Cada componente de un programa (módulo) se diseña y desarrolla de forma independiente y solo se combinan al final del proceso. Es la base de la **programación orientada a objetos**.
- **Programación declarativa**: El principio fundamental de esta programación es la descripción del resultado final. Por tanto, se centra el 'qué' antes que en el 'cómo'. Esto hace que el código sea más abstracto y difícil de comprender, aunque más corto y preciso. Dentro de este paradigma, también existen varios enfoques, siendo los más importantes la programación **funcional y lógica**. Los lenguajes de programación declarativa más conocidos son Prolog, Lisp, Haskell o SQL.
  - » **Programación funcional**: Se centra en el uso de funciones.
  - » **Programación lógica**: Se basa en lógica matemática.

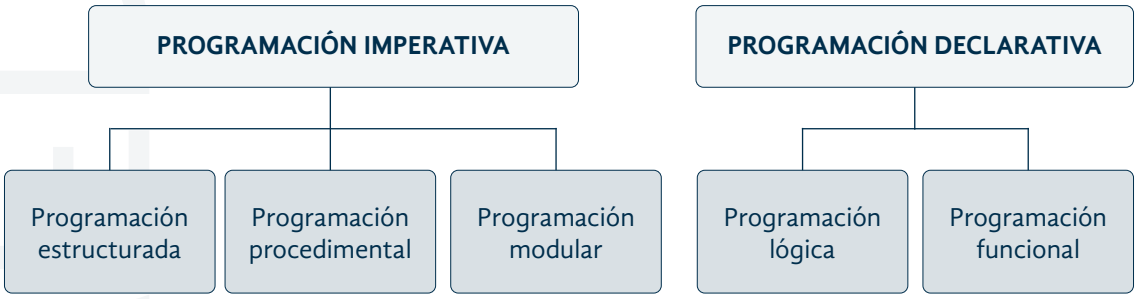

Fig.3. Paradigmas de programación.

Enlaces de interés...

Accede a este artículo para aprender más sobre los diferentes paradigmas de programación:
https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/paradigmas-de-programacion/

/ 6. Lenguajes de marcado

Por último, es conveniente hablar de otros lenguajes que, si bien carecen de algoritmos y no disponen de funciones aritméticas, también son muy útiles, especialmente a la hora de mostrar información en disposición digital.

Los **lenguajes de marcado** (*Markup Language*, en inglés) son un tipo de lenguaje de programación que permite la codificación de un texto estándar mediante un conjunto de símbolos o etiquetas que controlan la estructura, el formato del texto o las relaciones entre las diferentes partes que lo conforman.

Los lenguajes de marcado pueden ser interpretados por un dispositivo (impresora, navegador, página web) para controlar cómo debe verse un documento cuando se imprime o se muestra en una pantalla. Por lo tanto, un documento basado en lenguaje de marcado constará de dos tipos de textos: el texto que se mostrará y las etiquetas con la información.

Los lenguajes de marcado se dividen en tres tipos:

- **Lenguajes de presentación**: Se centran exclusivamente en controlar la apariencia del texto. Estas etiquetas son invisibles para el usuario. Un ejemplo sería **TeX**, que permite representar fórmulas matemáticas.

- **Lenguajes de procedimientos**: Orientados también a la presentación, pero, además, el programa que representa el documento debe interpretar las etiquetas para realizar acciones en función de ellas. Un ejemplo sería **HTML** (**Hypertext Markup Language**), que constituye el estándar general para la creación de páginas web.

- **Lenguajes descriptivos o semánticos**: Describen las diferentes partes o fragmentos en los que se estructura el documento, definiendo así su contenido, pero sin especificar cómo deben representarse. Un ejemplo de este tipo es **XML** (*Extensible Markup Language*), un lenguaje muy usado para intercambiar datos en la web.

```
<html>
        <head>
                <title>Mi página de ejemplo</title>
        </head>
        <body>
        Aquí va el contenido
        </body>
</html>
```

Fig.4. Ejemplo de código HTML.

Audio 1. "¿Qué es un lenguaje de marcado?"
https://bit.ly/3pdivnB

/ 7. Caso práctico 2: "La base del *web scraping*"

Planteamiento: Eres analista de datos en una empresa de telefonía y tu jefa te ha pedido que le prepares una comparativa de los planes para teléfonos móviles de la mayor empresa de la competencia.

Nudo: Una vez identificados los planes de la empresa competidora, llega el momento de extraer la información de las páginas de cada una de ellas. Podrías ir una por una apuntando en una hoja de cálculo los diferentes planes y precios, pero tardarías mucho. Además, no es una opción: tu jefa lo quiere para el final de la semana.

Desenlace: Esta tarea es idónea para poner en práctica tus habilidades para hacer '*web scraping*'. El *web scraping* es una técnica que sirve para extraer información de páginas web de forma automatizada. Básicamente, consiste en inspeccionar el código HTML en el que están basadas las páginas para identificar la información que se busca. ¿Cómo se identifica esta información? ¡Gracias a las etiquetas!

Por tanto, como analista de datos, tu trabajo consistirá en inspeccionar el código de las páginas donde se encuentran los planes para teléfonos móviles y sus respectivos precios, encontrar las etiquetas que engloban los datos que buscas y desarrollar un algoritmo de *web scraping* que acceda a esas páginas, busque las etiquetas de interés, extraiga la información contenida en ellas y las guarde en una hoja de cálculo o un archivo CSV.

Si fueran muchas páginas donde buscar, quizá podrías desarrollar un algoritmo de *web crawler*, que se diferencia del *web scraping* en que permite navegar de forma sistémica por diferentes páginas web.

Por ello, aunque los lenguajes de marcado se asocian más a los desarrollados webs, es conveniente que los analistas y científicos de datos tengan unas nociones básicas de lenguajes como HTML, ya que son la base para poder diseñar los algoritmos de *web scraping* y *web crawling*.

Enlaces de interés...

A continuación, en este enlace, podrás conocer las diferencias entre *web scraping* y *web crawling*: https://openwebinars.net/blog/diferencias-entre-scraping-crawling-y-parsing/

/ 8. Resumen y resolución del caso práctico de la unidad

En esta unidad, hemos analizado diferentes tipologías para clasificar los lenguajes de programación. En primer lugar, hemos estudiado qué son los lenguajes de bajo y alto nivel, identificando sus diferencias y dando varios ejemplos de cada tipo.

En segundo lugar, hemos abordado la clasificación de los lenguajes en función del paradigma de programación en el que están inspirados. Esencialmente, podemos distinguir dos paradigmas: programación imperativa y declarativa.

Por último, hemos hecho una introducción a los lenguajes de marcado, que permiten controlar el formato, la estructura y la relación de los textos en el mundo digital.

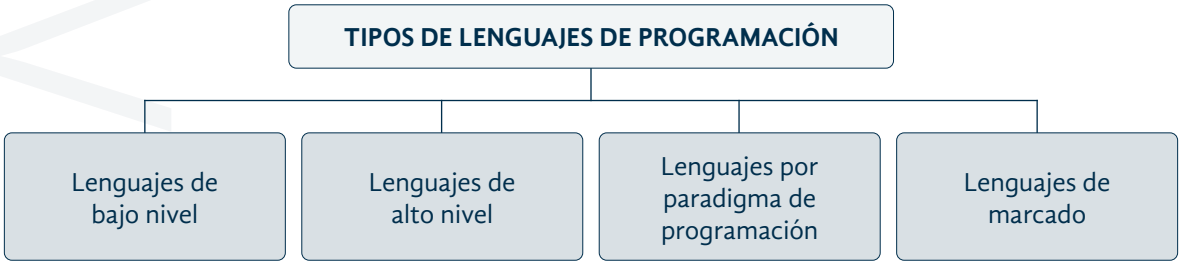

Fig.5. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Independientemente de que se trate de un videojuego, un coche teledirigido o una *app* de radio, una decisión crucial que deben tomar los programadores antes de escribir la primera línea de código es elegir el lenguaje o los lenguajes de programación que van a utilizar para desarrollar su software.

Existen muchísimos lenguajes de programación, todos ellos basados en la programación imperativa o la declarativa. La elección del lenguaje de programación va a condicionar el futuro del proyecto en muchos aspectos, desde el uso de entornos de desarrollo hasta la contratación de desarrolladores.

Por tanto, es importante tener en cuenta las particularidades de cada lenguaje y las ventajas y los inconvenientes de cada uno. En el caso de la *app* de móvil, sería interesante **investigar qué lenguajes son más populares y eficaces para el desarrollo de *apps***. Además, habrá que tener en cuenta para qué **tipo de dispositivo** se quiere desarrollar la *app*. Así, en el caso de ser una aplicación para Android, los lenguajes más populares son Java y Kotlin, mientras que para iOS, los más utilizados son Bbjective C y Swift.

/ 9. Bibliografía

Yanez, C. (2021). *¿Qué es un lenguaje de programación y qué tipos existen?* Deusto Formación. Disponible en: https://www.deustoformacion.com/blog/programacion-tic/que-es-lenguaje-programacion-que-tipos-existen

Javapoint. *What is a programming language?* Disponible en: https://www.javatpoint.com/classification-of-programming-languages

Rockcontent (2019) *¿Qué es un lenguaje de programación y qué tipos existen?* Disponible en: https://rockcontent.com/es/blog/que-es-un-lenguaje-de-programacion/

Morales, R. (2018). *Qué son los lenguajes de marcas.* Ticarte. Disponible en: https://www.ticarte.com/contenido/que-son-los-lenguajes-de-marcas

Ionos (2020). *Paradigmas de programación: principios básicos de programación.* Disponible en: https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/paradigmas-de-programacion/

MEDAC